

ДОМАШНЕЕ ЗАДАНИЕ 1: УДАЛЕНИЕ ЗАМЕТКИ

Сейчас приложение умеет выводить список заметок. Нужно сделать так, чтобы рядом с каждой заметкой была кнопка удаления, а после нажатия эта заметка исчезала из списка.

Ссылки на исходники проектов для ДЗ для групп:

ПД-412 <https://drive.google.com/file/d/1nut6vQSI5p7gwkxeS7fuoVES8SJ8KbL/view?usp=sharing>

П-318 <https://drive.google.com/file/d/17ZEUocMoqZTFsOpHSh6e2VgfNzTU8h52/view?usp=sharing>

П-319 <https://drive.google.com/file/d/13ud8MvYOfLjBBZkHwT9S4bSpDBWvRPPR/view?usp=sharing>

1.1 Ход работы

1.1.1 Добавить форму удаления в partial заметки

Файл [Pages/Shared/_Note.cshtml](#)

Внутри разметки заметки нужно добавить маленькую форму с кнопкой удаления.

В Razor Pages любая модификация данных с клиентской стороны выполняется либо через JS fetch-запрос, либо через форму с методом POST. Поскольку удаление элемента — это модификация данных, то мы воспользуемся вторым способом, который был разобран на практическом занятии, так как в данном сценарии он проще и не требует никакой дополнительной JS-логики.

Данная форма будет отличаться от формы создания заметки. В форме создания использовалось свойство модели с атрибутом `[BindProperty]`, однако для удаления мы должны использовать уже имеющуюся у нас запись, из которой нас интересует только свойство `public int Id { get; set; }`. Поля `Id` достаточно, чтобы однозначно идентифицировать элемент в списке. Непосредственно для обработки удаления на стороне сервера в классе модели

IndexModel мы создадим вот такой метод с суффиксом Delete и одним входным параметром id.

```
public IActionResult OnPostDelete(int id)
{
    // найти заметку по id
    // удалить ее
    // вернуть пользователя обратно на страницу
    return RedirectToPage();
}
```

Чтобы форма смогла передать параметр id в обработчик необходимо создать специальный невидимый <input>, атрибут value которого будет хранить как раз таки id текущей заметки. <input type="hidden" name="id" value="..." />. В качестве value в input должен быть передан id текущей заметки. В противном случае форма не сможет отправить id заметки в обработчик.

Чтобы форма отправилась, в нее, необходимо добавить кнопку с типом submit. Таким образом, у нас получится форма, содержащая в себе скрытый input и кнопку submit. У самой формы необходимо указать помимо метода еще атрибут asp-page-handler, чтобы при нажатии на кнопку данные из формы обрабатывал нужный обработчик.

1.1.2 Добавить в интерфейс сервиса заметок новый метод

Поскольку хранение заметок мы реализовали с помощью DI, то нам необходимо добавить новый метод, который позволит удалять заметку по ее id. Для этого в интерфейс INoteService необходимо добавить еще один метод

```
public interface INoteService
{
    public List<Note> GetAll();
    public void Add(EditableNote editable);
    public void Delete(int id);
}
```

После этого необходимо реализовать метод интерфейса в классах InMemoryNoteService и SqliteNoteService. В InMemoryNoteService требуется добавить полностью рабочую реализацию с поиском по id и удалением из списка (не забудьте проверить, что элемента с таким id может и не быть в

списке). В `SqliteNoteService` достаточно добавить просто заглушку. Реализация данного сервиса – это тема следующего занятия.

1.2 Добавить логику для авто генерации id заметки в доменной модели

Если вы скачали к себе проект, который был создан на практическом занятии, и выполнили все описанные выше шаги, то, скорее всего, при попытке удаления конкретной заметки, в зависимости от вашей реализации алгоритма поиска, у вас вместо необходимой заметки удаляется первая или последняя, либо удаление не работает вовсе. Это связано с тем, что мы убрали из доменной модели логику для авто генерации id, так как в дальнейшем вместо нее мы будем использовать id, сгенерированный базой данных. Чтобы все работало и в `InMemoryNoteService` и в `SqliteNoteService`, нам необходимо реализовать отдельную логику генерации id заметки прямо в `InMemoryNoteService`. Для этого необходимо повторить ту же логику, что была раньше в классе `Note`: создать в `InMemoryNoteService` статическое приватное поле `_idCounter`. И при добавлении новой заметки в список инкрементировать этот `_idCounter` и присваивать его в `Note.Id`. ВАЖНО: нельзя использовать длину списка в качестве id заметки, так как длина может сокращаться при удалении записи.

1.3 Требования к результату

После выполнения задания:

- 1) У каждой заметки есть кнопка удаления.
- 2) Кнопка отправляет POST-запрос.
- 3) В обработчик передается id выбранной заметки.
- 4) Удаление выполняется через `INoteService`.
- 5) Заметка исчезает из списка.
- 6) При добавлении новой заметки через `InMemoryNoteService` ей присваивается уникальный Id.

7) Приложение не падает, если заметка с указанным id не найдена (передать невалидный id можно, подменив его в коде страницы с id модели на, например, -1)

1.4 Вопросы для самопроверки

Вопросы предназначены исключительно для самоконтроля, присылать их вместе с домашним заданием не нужно

- 1) Почему после удаления используется RedirectToPage()?
- 2) Как asp-page-handler="Delete" связан с методом OnPostDelete?
- 3) Зачем в форме удаления нужен скрытый input?